

CSE321 – Lab Session #2

Relational Schema Design and Normalization Report

Aliona Pirozhenko (20232008)

UNIST

South Korea

alionapirozhenko@unist.ac.kr

1 Introduction

The purpose of this report is to critically document, review, and correct the relational schema I designed during Lab Session #2 of CSE321 Database Systems. The lab required me to convert a given Entity-Relationship (ER) diagram [2] into a relational database schema [3], identify functional dependencies (FDs) [1] for each relation, determine candidate keys and primary keys, and verify whether each relation satisfies Third Normal Form (3NF) [4].

This report is structured as follows. Section 2 presents the exact schema I submitted during the in-class activity together with an honest reconstruction of my reasoning at the time. Section 3 presents the fully corrected schema, with precise specifications of attributes, keys, FDs, and 3NF (and where applicable, BCNF [5]) analysis for each relation. Section 4 explains, for each modification, what was wrong in my original design, why it violates relational design principles [6], and how the revised schema resolves the issue. Section 5 summarises the lessons learned.

The ER diagram distributed during the lab described a university database system covering entities including Person, Staff, Laboratory, Equipment, Student, Course, Professor, and

Stakeholder, with ISA hierarchies [6], several relationships, and cardinality constraints. The full ER diagram is shown in Figure 1.

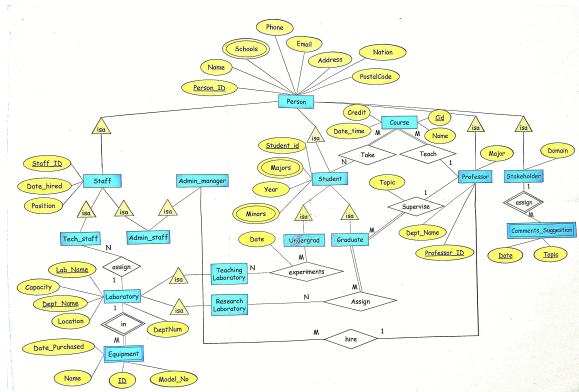


Figure 1: ER Diagram as provided during Lab Session #2.

2 Review of the Original Design

2.1 Initial Notes and Definitions I Wrote

I wrote two “non-trivial” FD examples as a warm-up reference:

$$A \rightarrow B \quad \text{and} \quad AC \rightarrow BC$$

Looking back at this, I believe I was trying to ground myself in FD notation before diving into the ER diagram. The second example,

$AC \rightarrow BC$, is simply the augmentation of $A \rightarrow B$ by adding C to both sides — it is a derived rule, not an independently non-trivial FD. At the time I likely recalled it as a useful identity, but calling it “non-trivial” was imprecise. It reflects that I was still settling into the FD formalism at the start of the lab.

2.2 The Schema I Submitted

Below I reproduce, verbatim, every relation I submitted during the lab session, exactly as I wrote them on paper.

(1) Person.

Person(Person_ID, Name, Schools, Phone, Email, Address, Nation, PostalCode)

Keys: {Person_ID, Name}, {Person_ID, Phone}, {Person_ID, Schools}, {Person_ID, Email}, {Person_ID, Address}, {Person_ID, Nation}, {Person_ID, PostalCode}

PK: {Person_ID}

FDs: Person_ID $\rightarrow$ Name, Schools, Phone, Email, Address, Nation, PostalCode

Conclusion: Is in 3NF

(2) Staff.

Staff(Staff_ID, Date_hired, Position, Type)

Keys: {Staff_ID, Date_hired}, {Staff_ID, Position}, {Staff_ID, Type}

PK: {Staff_ID}

FDs: Staff_ID $\rightarrow$ Date_hired, Position, Type

Conclusion: Is in 3NF

(3) Laboratory.

Laboratory(Lab_name, Dept_Name, Capacity, Location, DeptNum, Equipment_ID)

Keys: {Lab_Name, Dept_Name, Capacity}, {Lab_Name, Dept_Name, Location}, {Lab_Name, Dept_Name, DeptNum}

PK: {Lab_Name, Dept_Name}

FDs: Lab_Name, Dept_Name $\rightarrow$ Capacity, Location, DeptNum, Equipment_ID

Conclusion: Not in 3NF

(4) Equipment.

Equipment(ID, Model_No, Date_Purchased, Name, Dept_Name, Lab_Name)

Keys: {ID, Dept_Name, Lab_Name, Model_No},
 {ID, Dept_Name, Lab_Name, Name},
 {ID, Dept_Name, Lab_Name, Date_Purchased}
 PK: {ID, Dept_Name, Lab_Name}
 FDs: ID, Dept_Name, Lab_Name → Model_No,
 Date_Purchased, Name

Conclusion: Not in 3NF

(5) *Student*.

Student(Student_ID, Majors, Minors, Year, Type,
 Course_list)

Keys: {Student_ID, Majors}, {Student_ID, Year},
 {Student_ID, Minors}, {Student_ID, Type},
 {Student_ID, Course_list}
 PK: {Student_ID}
 FDs: Student_ID → Majors, Minors, Year, Type,
 Course_list

Conclusion: Is in 3NF

(6) *Course*.

Course(Cid, Name, Credit, Date_time)

Keys: {Cid, Name}, {Cid, Credit}, {Cid, Date_time}
 PK: {Cid}
 FDs: Cid → Name, Credit, Date_time
 Conclusion: Is in 3NF

(7) *Professor*.

Professor(Professor_ID, Major, Dept_Name,
 teaching_courses, supervising_topics)

Keys: {Professor_ID, Major}, {Professor_ID, Dept_Name}
 PK: {Professor_ID}
 FDs: Professor_ID → Major, Dept_Name,
 teaching_courses, supervising_topics
 Conclusion: Is in 3NF

(8) *Stakeholder*.

Stakeholder(Person_ID, Domain,
 Comments_suggestion_date,
 comments_suggestion_topic)

Keys: {Person_ID, Domain}
 PK: {Person_ID}
 FDs: Person_ID → Domain, Comments_suggestion_date,
 comments_suggestion_topic
 Conclusion: (none stated)

2.3 My Reasoning at the Time

I will now reconstruct the logic I was applying for each relation, as honestly as I can.

Person: This was the most straightforward relation. I read the ER diagram directly and listed all attributes. My FD reasoning was correct: Person_ID is the sole identifier and determines everything else. Where I went wrong — and I can see this clearly now — is in the candidate key listing. I listed pairs like {Person_ID, Name} as candidate keys, which is wrong because they are not minimal. I

think I confused “candidate key” with “any set that contains the key,” i.e., I was listing superkeys. The logic was: “Person_ID is the PK and the other attributes describe the person, so combining them must also form a key.” That is true, but those combinations are superkeys, not candidates.

Staff: My main shortcut here was collapsing the ISA hierarchy into a single Type attribute. Under time pressure, rather than thinking through separate subtype tables, I reasoned: “Tech_Staff and Admin_Staff are just types of staff — I can store that distinction as an attribute.” This is a common simplification and understandable, but it loses the subtype-specific attributes (e.g., which laboratory a tech staff member is assigned to). The FDs are internally consistent, and the same candidate-key error repeats.

Laboratory: I correctly identified the composite PK {Lab_Name, Dept_Name} from the ER diagram, which marks Laboratory as having a two-part primary key. I also correctly included DeptNum as an attribute. However, I included Equipment_ID in this relation, which I now recognise was a direction error: since one Laboratory has many Equipment items, the foreign key must live on the Equipment side, not here. I flagged this relation as not in 3NF, which ends up being debatably correct depending on assumptions, but my stated reasoning was absent — I just wrote the conclusion without justification.

Equipment: I correctly identified this as a weak entity with the composite PK {ID, Dept_Name, Lab_Name}. The FDs I wrote are: {ID, Dept_Name, Lab_Name} → {Model_No, Date_Purchased, Name}. All non-key attributes depend on the full key with no partial or transitive dependencies. Ironically, given those FDs, this relation is actually in 3NF (and even in BCNF). I wrote “Not in 3NF” which is incorrect. I believe I flagged it incorrectly because I assumed weak entity structures automatically have normalization problems — which is a misconception.

Student: The fundamental error here is Course_list: I stored a list of courses a student is enrolled in as a single attribute. A relation containing a set-valued or list-valued attribute violates First Normal Form (1NF). A relation not in 1NF cannot be in 3NF. So my conclusion of “Is in 3NF” is incorrect at its foundation. I think I included Course_list because I was trying to capture the student-course relationship without creating an additional junction table — again, a time-pressure shortcut.

Course: The stated FDs and PK are internally consistent and the 3NF conclusion is correct *for what I wrote*. However, I missed capturing the relationship: a Course is taught by exactly one Professor. That $M : 1$ relationship from Course to Professor should be encoded as a foreign key Professor_ID within the Course relation. Without it, the “taught by” relationship is entirely lost from my schema.

Professor: I included teaching_courses and supervising_topics as attributes. Both are multi-valued (a professor teaches many courses, supervises many students on different topics), which again violates 1NF, making the 3NF claim invalid. I also added Dept_Name to Professor, which was not in the ER diagram and introduces an unjustified attribute. The listed candidate keys {Professor_ID, Major} and {Professor_ID, Dept_Name} are superkeys, not candidate keys.

Stakeholder: I embedded Comments_suggestion_date and

comments_suggestion_topic directly in the Stakeholder relation. The ER diagram explicitly shows Comments_Suggestion as a *weak entity* of Stakeholder — it should be a separate relation. By embedding it, I implicitly modelled a one-to-one relationship between stakeholders and comments, when in reality one stakeholder can submit many comments over time. I also did not state a 3NF conclusion, which means I may have recognised something was off but did not resolve it.

3 Revised Design

3.1 Assumptions

Before presenting the revised schema, I state all assumptions explicitly, since the ER diagram leaves some semantic constraints unspecified.

- A1. Person_ID is a globally unique identifier** shared across all ISA subtypes (Staff, Student, Professor, Stakeholder). Every subtype relation stores Person_ID as a foreign key referencing Person.
- A2. A Laboratory is uniquely identified by (Lab_Name, Dept_Name).**
Two laboratories in different departments may share the same name, making the composite key necessary. This is consistent with the ER diagram marking both as primary key attributes.
- A3. Dept_Name functionally determines DeptNum**, i.e., each department has exactly one department number, and each number belongs to exactly one department. This is a natural real-world assumption. It leads to a 3NF (and BCNF) violation in Laboratory that requires decomposition.
- A4. Equipment ID is unique only within a laboratory.** Since Equipment is a weak entity of Laboratory, its partial key ID uniquely identifies a piece of equipment only within the context of a specific laboratory instance.
- A5. A course is offered at a fixed time and has a single instructor.** The M courses taught by 1 professor constraint from the ER diagram is encoded as a Professor_ID foreign key in Course.
- A6. Comments_Suggestion is identified by (Person_ID, Date, Time).** The ER diagram states Date as a partial key; I additionally include Time to handle the realistic case where one stakeholder submits multiple comments on the same date.
- A7. The ISA hierarchies are modelled using the table-per-subclass strategy.** Each subtype is a separate relation whose PK is a foreign key to the parent relation.
- A8. A professor's Major is a single-valued attribute.** This matches the ER diagram, which lists it as a plain attribute of Professor.

3.2 Normal Form Criteria (Reference)

For clarity, I briefly restate the relevant normal form definitions used throughout this section.

1NF: Every attribute value is atomic (no multi-valued or set-valued attributes).

2NF: In 1NF, and no non-prime attribute is partially dependent on any candidate key (relevant only when a candidate key has more than one attribute).

3NF: In 2NF, and no non-prime attribute is transitively dependent on any candidate key. Equivalently: for every non-trivial FD $X \rightarrow A$, either X is a superkey, or A is a prime attribute (member of some candidate key).

BCNF (Boyce-Codd Normal Form): Stricter than 3NF. For every non-trivial FD $X \rightarrow A$, X must be a superkey. BCNF does not have the prime-attribute exception that 3NF has.

3.3 Revised Relations

For each relation I list: attributes, candidate keys, PK, foreign keys (FKs), functional dependencies, and a normal form analysis.

3.3.1 Person.

Person(Person_ID, Name, Schools, Phone,
Email, Address, Nation, PostalCode)

Candidate Keys: {Person_ID}

PK: {Person_ID}

FKs: (none)

FDs: Person_ID $\rightarrow$ Name, Schools, Phone, Email,
Address, Nation, PostalCode

1NF: All attributes are atomic. ✓

Candidate keys: Only {Person_ID} is a candidate key. All other listed sets in my original (e.g., {Person_ID, Name}) are superkeys — they are not minimal.

2NF: The only candidate key is a single attribute, so partial dependency cannot occur. ✓

3NF: The only non-trivial FD has Person_ID (a superkey) on the LHS. No transitive dependencies exist. ✓

BCNF: Same reasoning — the only non-trivial FD has a superkey on the LHS. ✓ *Person is in BCNF.*

3.3.2 Staff.

Staff(Staff_ID, Person_ID, Date_hired, Position)

Candidate Keys: {Staff_ID}, {Person_ID}

PK: {Staff_ID}

FKs: Person_ID $\rightarrow$ Person(Person_ID)

FDs: Staff_ID $\rightarrow$ Person_ID, Date_hired, Position

Person_ID $\rightarrow$ Staff_ID, Date_hired, Position

1NF: All attributes are atomic. ✓

Candidate keys: Both {Staff_ID} and {Person_ID} are minimal keys — each alone determines all other attributes.

2NF: All candidate keys are single attributes, so no partial dependency can occur. ✓

3NF: Every non-trivial FD has a candidate key on the LHS. ✓

BCNF: Every non-trivial FD has a superkey on the LHS. ✓ *Staff is in BCNF.*

Note: The Type attribute from my original design is removed. Subtype distinctions (Tech_Staff, Admin_Staff) are handled by separate subtype relations below.

3.3.3 Tech_Staff.

Tech_Staff(Staff_ID, Lab_Name, Dept_Name)

Candidate Keys: {Staff_ID}

PK: {Staff_ID}

FKs: Staff_ID → Staff(Staff_ID)
 (Lab_Name, Dept_Name) → Laboratory(Lab_Name, Dept_Name)
 FDs: Staff_ID → Lab_Name, Dept_Name

Rationale: The ER diagram states N Tech_Staff are assigned to 1 Laboratory. From the tech-staff side, each staff member is assigned to exactly one lab, so Lab_Name and Dept_Name are stored as attributes here (the FK lives on the “many” side of the relationship).

3NF: Only FD has Staff_ID (superkey) on LHS. ✓

BCNF: Same. ✓ *Tech_Staff is in BCNF.*

3.3.4 Admin_Staff.

Admin_Staff(Staff_ID)

Candidate Keys: {Staff_ID}
 PK: {Staff_ID}
 FKs: Staff_ID → Staff(Staff_ID)
 FDs: (none beyond the key itself)

3NF / BCNF: No non-trivial FDs. Trivially in BCNF. ✓

3.3.5 Admin_Manager. The ER diagram states that Admin_Manager is a subtype of Admin_Staff, and that 1 Professor hires M Admin_Managers.

Admin_Manager(Staff_ID, Hired_by_Professor_ID)

Candidate Keys: {Staff_ID}
 PK: {Staff_ID}
 FKs: Staff_ID → Admin_Staff(Staff_ID)
 Hired_by_Professor_ID → Professor(Professor_ID)
 FDs: Staff_ID → Hired_by_Professor_ID

3NF: The only non-trivial FD has Staff_ID (superkey) on LHS. ✓

BCNF: Same. ✓ *Admin_Manager is in BCNF.*

3.3.6 Department. This relation is introduced as a result of decomposing Laboratory to eliminate the 3NF violation caused by assumption A3 ($\text{Dept_Name} \rightarrow \text{DeptNum}$).

Department(Dept_Name, DeptNum)

Candidate Keys: {Dept_Name}, {DeptNum}
 PK: {Dept_Name}
 FKs: (none)
 FDs: Dept_Name → DeptNum
 DeptNum → Dept_Name

3NF: Both non-trivial FDs have candidate keys on the LHS. ✓

BCNF: Both non-trivial FDs have superkeys on the LHS. ✓
Department is in BCNF.

3.3.7 Laboratory. After decomposing out DeptNum into Department, and removing the incorrectly placed Equipment_ID, the cleaned relation is:

Laboratory(Lab_Name, Dept_Name, Capacity,
 Location, Lab_Type)

Candidate Keys: {Lab_Name, Dept_Name}
 PK: {Lab_Name, Dept_Name}
 FKs: Dept_Name → Department(Dept_Name)
 FDs: Lab_Name, Dept_Name → Capacity, Location, Lab_Type

Lab_Type is an attribute encoding the ISA specialisation (Teaching or Research). The subtype-specific attributes and relationships are handled in separate relations below.

1NF: All attributes are atomic. ✓

2NF: Capacity, Location, and Lab_Type all depend on the *full* composite key {Lab_Name, Dept_Name}. Neither attribute is determined by Lab_Name alone or Dept_Name alone. No partial dependency. ✓

3NF: The only non-trivial FD has the PK (a superkey) on the LHS. No transitive dependencies among non-key attributes. ✓

BCNF: Same reasoning. ✓ *Laboratory is in BCNF.*

3.3.8 Teaching_Laboratory.

Teaching_Laboratory(Lab_Name, Dept_Name,
 Teaching_Date)

Candidate Keys: {Lab_Name, Dept_Name}
 PK: {Lab_Name, Dept_Name}
 FKs: (Lab_Name, Dept_Name) →
 Laboratory(Lab_Name, Dept_Name)
 FDs: Lab_Name, Dept_Name → Teaching_Date

3NF / BCNF: Only non-trivial FD has the PK (superkey) on LHS. ✓
Teaching_Laboratory is in BCNF.

3.3.9 Research_Laboratory.

Research_Laboratory(Lab_Name, Dept_Name)

Candidate Keys: {Lab_Name, Dept_Name}
 PK: {Lab_Name, Dept_Name}
 FKs: (Lab_Name, Dept_Name) →
 Laboratory(Lab_Name, Dept_Name)
 FDs: (none beyond the key itself)

3NF / BCNF: No non-trivial FDs. Trivially in BCNF. ✓

3.3.10 Equipment. Equipment is a weak entity of Laboratory. Its full identifier is the partial key ID combined with the identifying owner key {Lab_Name, Dept_Name}.

Equipment(ID, Lab_Name, Dept_Name,
 Model_No, Date_Purchased, Name)

Candidate Keys: {ID, Lab_Name, Dept_Name}
 PK: {ID, Lab_Name, Dept_Name}
 FKs: (Lab_Name, Dept_Name) →
 Laboratory(Lab_Name, Dept_Name)
 FDs: ID, Lab_Name, Dept_Name →
 Model_No, Date_Purchased, Name

1NF: All attributes are atomic. ✓

2NF: Are there partial dependencies? We must check whether any non-key attribute is determined by a proper subset of the PK.

- ID alone $\nrightarrow$ Model_No, Date_Purchased, Name (the same ID number could exist in a different lab).
- {Lab_Name, Dept_Name} alone $\nrightarrow$ any non-key attribute.

No partial dependencies. ✓

3NF: The only non-trivial FD has the full PK (a superkey) on LHS. No transitive dependencies. ✓

BCNF: Same. ✓ *Equipment is in BCNF.*

This corrects my original error of flagging Equipment as not in 3NF. Given the FDs I myself wrote, the relation was and remains correctly normalised.

3.3.11 Student.

Student(Student_ID, Person_ID, Majors,
Minors, Year, Student_Type)

Candidate Keys: {Student_ID}, {Person_ID}
PK: {Student_ID}
FKs: Person_ID → Person(Person_ID)
FDs: Student_ID → Person_ID, Majors, Minors,
Year, Student_Type
Person_ID → Student_ID, Majors, Minors,
Year, Student_Type

Course_list is removed (see Revision 4.6). Student_Type (Undergrad or Graduate) is retained as an atomic attribute encoding the ISA distinction.

1NF: All attributes are atomic. ✓
3NF: All non-trivial FDs have a candidate key on the LHS. ✓
BCNF: All non-trivial FDs have a superkey on the LHS. ✓ *Student is in BCNF.*

3.3.12 Undergrad.

Undergrad(Student_ID)

Candidate Keys: {Student_ID}
PK: {Student_ID}
FKs: Student_ID → Student(Student_ID)
FDs: (none beyond the key itself)
3NF / BCNF: No non-trivial FDs. Trivially in BCNF. ✓

3.3.13 Graduate. A graduate student is assigned to a Research Laboratory and supervised by a Professor on a specific topic.

Graduate(Student_ID, Lab_Name, Dept_Name,
Supervisor_Professor_ID, Topic)

Candidate Keys: {Student_ID}
PK: {Student_ID}
FKs: Student_ID → Student(Student_ID)
(Lab_Name, Dept_Name) →
Research_Laboratory(Lab_Name, Dept_Name)
Supervisor_Professor_ID →
Professor(Professor_ID)
FDs: Student_ID → Lab_Name, Dept_Name,
Supervisor_Professor_ID, Topic
3NF: Only non-trivial FD has Student_ID (superkey) on LHS. ✓
BCNF: Same. ✓ *Graduate is in BCNF.*

3.3.14 Professor.

Professor(Professor_ID, Person_ID, Major)

Candidate Keys: {Professor_ID}, {Person_ID}
PK: {Professor_ID}
FKs: Person_ID → Person(Person_ID)
FDs: Professor_ID → Person_ID, Major

Person_ID → Professor_ID, Major

Dept_Name, teaching_courses, and supervising_topics are all removed (see Revisions 4.3 and 4.10).

3NF: All non-trivial FDs have a candidate key on LHS. ✓
BCNF: All non-trivial FDs have a superkey on LHS. ✓ *Professor is in BCNF.*

3.3.15 Course.

Course(Cid, Name, Credit, Date_time,
Professor_ID)

Candidate Keys: {Cid}
PK: {Cid}
FKs: Professor_ID → Professor(Professor_ID)
FDs: Cid → Name, Credit, Date_time, Professor_ID

Professor_ID is added as a FK encoding the “M Courses taught by 1 Professor” relationship from the ER diagram.

1NF: All attributes are atomic. ✓
3NF: The only non-trivial FD has Cid (a superkey) on LHS. ✓
BCNF: Same. ✓ *Course is in BCNF.*

3.3.16 Takes (Student–Course Relationship). The ER diagram specifies an $M : N$ relationship between Student and Course (double-lined, meaning students have total participation — every student must take at least one course). A many-to-many relationship requires its own relation.

Takes(Student_ID, Cid)

Candidate Keys: {Student_ID, Cid}
PK: {Student_ID, Cid}
FKs: Student_ID → Student(Student_ID)
Cid → Course(Cid)
FDs: (none beyond the key itself)

1NF: All attributes are atomic. ✓
2NF: The PK is composite. Are there partial dependencies?
• Student_ID alone does not determine Cid (a student takes multiple courses).
• Cid alone does not determine Student_ID (a course has multiple students).

No partial dependencies. ✓

3NF / BCNF: No non-trivial FDs beyond the key. Trivially in BCNF. ✓

3.3.17 Uses (Undergrad–Teaching_Laboratory Relationship). The ER diagram specifies N Teaching Laboratories used for experiments by M Undergraduates — another many-to-many relationship.

Uses(Student_ID, Lab_Name, Dept_Name)

Candidate Keys: {Student_ID, Lab_Name, Dept_Name}
PK: {Student_ID, Lab_Name, Dept_Name}
FKs: Student_ID → Undergrad(Student_ID)
(Lab_Name, Dept_Name) →
Teaching_Laboratory(Lab_Name, Dept_Name)
FDs: (none beyond the key itself)

2NF: We check partial dependencies. Does any proper subset of {Student_ID, Lab_Name, Dept_Name} determine any non-key

attribute? There are *no non-key attributes*, so this is trivially satisfied. ✓

3NF / BCNF: No non-trivial FDs. Trivially in BCNF. ✓

3.3.18 Stakeholder.

Stakeholder(Person_ID, Domain)

Candidate Keys: {Person_ID}

PK: {Person_ID}

FKs: Person_ID → Person(Person_ID)

FDs: Person_ID → Domain

3NF: The only non-trivial FD has Person_ID (a superkey) on LHS. ✓

BCNF: Same. ✓ *Stakeholder is in BCNF.*

3.3.19 Comments_Suggestion. Comments_Suggestion is a weak entity of Stakeholder. Its partial key is {Date, Time}, and its identifying owner is Stakeholder via Person_ID.

Comments_Suggestion(Person_ID, Date, Time, Content)

Candidate Keys: {Person_ID, Date, Time}

PK: {Person_ID, Date, Time}

FKs: Person_ID → Stakeholder(Person_ID)

FDs: Person_ID, Date, Time → Content

Content captures the body of the comment or suggestion — a reasonable assumption for what this weak entity stores.

1NF: All attributes are atomic. ✓

2NF: We check whether any non-key attribute (Content) depends on a proper subset of the PK:

- Person_ID alone $\nrightarrow$ Content (one person may make many different comments).
- {Date, Time} alone $\nrightarrow$ Content (comments at the same time could come from different stakeholders).

No partial dependency. ✓

3NF: The only non-trivial FD has the full PK (a superkey) on LHS. ✓

BCNF: Same. ✓ *Comments_Suggestion is in BCNF.*

4 Revision Details

I now go through each mistake in my original design systematically, explaining what was wrong, why it violates relational design principles, and how the revised schema resolves it.

4.1 Revision 1: Superkeys Listed as Candidate Keys (All Relations)

What was wrong: In every relation, I listed compound sets such as {Person_ID, Name}, {Staff_ID, Date_hired}, or {Professor_ID, Major} as candidate keys. A candidate key must be *minimal*: no proper subset of it is also a key. Since Person_ID alone already determines all attributes in Person, any set containing Person_ID along with additional attributes is a superkey, not a candidate key.

Why it matters: The entire 3NF (and BCNF) analysis depends on correctly identifying candidate keys. Specifically, whether a non-trivial FD $X \rightarrow A$ violates 3NF depends on whether X is a superkey and whether A is prime (part of some candidate key).

With wrong candidate keys, the prime-attribute classification is also wrong, invalidating all conclusions.

Why I made this mistake: I believe I conflated the concept of “key” with “any set of attributes that contains the primary key.” Since I was listing the PK first and then combining it with other attributes, I may have been thinking: “these attributes uniquely identify a tuple, so they must be a key.” That reasoning correctly identifies uniqueness but ignores minimality.

Fix: I carefully verified minimality for each candidate key in the revised schema. For single-attribute PKs (Person, Staff, Equipment within its lab, Student, Course, Professor, Stakeholder), the only candidate key is the single identifying attribute. Where a second candidate key exists (e.g., Person_ID in Staff because each staff member is also a unique person), I included it explicitly.

4.2 Revision 2: ISA Hierarchies Collapsed into a Type Attribute

What was wrong: For both Staff and Student, I collapsed the ISA hierarchy into a single Type attribute (e.g., Tech_Staff vs. Admin_Staff stored as a string). While this correctly records the subtype category, it entirely loses subtype-specific attributes and relationships.

For example, Tech_Staff is assigned to a specific Laboratory, and Graduate students are assigned to a Research_Laboratory and supervised by a Professor on a topic. None of this information can be captured by a Type column alone.

Why it matters: This is a semantic incompleteness error. The schema fails to represent the full content of the ER diagram. Foreign key relationships to subtypes (e.g., tech staff to laboratory) are dropped entirely.

Why I made this mistake: Under time pressure, this was the fastest way to “include” the ISA information without drawing up separate tables. The reasoning in my head was: “I know there are subtypes — I can just record which type each person is.” This is not inherently wrong in a denormalised design, but it omits the relational content of the subtypes.

Fix: I applied the *table-per-subclass* strategy.

Each subtype (Tech_Staff, Admin_Staff, Admin_Manager, Undergrad, Graduate, Teaching_Laboratory,

Research_Laboratory) is its own relation whose primary key is a foreign key to the parent relation. Subtype-specific attributes and relationships are placed in the correct subtype table.

4.3 Revision 3: Multi-valued Attributes Violating 1NF

What was wrong: I included the following non-atomic attributes:

- Course_list in Student — a list of courses a student takes.
- teaching_courses in Professor — a list of courses taught.
- supervising_topics in Professor — a list of (student, topic) pairs for graduate supervision.

All three violate First Normal Form (1NF), which requires every attribute to be atomic.

Why it matters: A relation that is not in 1NF cannot be in 2NF or 3NF — all higher normal forms presuppose 1NF. My conclusions of “Is in 3NF” for Student and Professor are therefore invalid

at their foundation. Beyond normalization, non-atomic attributes make SQL operations (filtering, joining, aggregating) extremely difficult or impossible to express cleanly.

Why I made this mistake: I was trying to capture relationships without introducing additional tables. For `Course_list`, I thought: “a student’s courses are part of their profile, so I should include them here.” For `Professor`, I thought the same. These were natural intuitions but they reflect a misunderstanding of how many-to-many relationships must be modelled relationally.

Fix:

- `Course_list` is removed from `Student`. The $M : N$ student-course relationship is modelled as a separate `Takes(Student_ID, Cid)` relation.
- `teaching_courses` is removed from `Professor`. The relationship is captured by adding `Professor_ID` as a FK in `Course` (the $M : 1$ “taught by” relationship, where each course has exactly one professor).
- `supervising_topics` is removed from `Professor`. The supervision relationship is captured in `Graduate(Student_ID, ..., Supervisor_Professor_ID, Topic)`, where each graduate student has one supervisor and one topic.

4.4 Revision 4: Equipment_ID Placed in Laboratory

What was wrong: I included `Equipment_ID` as an attribute of `Laboratory`. The ER diagram specifies a $1 : M$ relationship from `Laboratory` to `Equipment` (one laboratory has many pieces of equipment). In relational modelling, a $1 : M$ relationship is encoded by placing the foreign key on the “many” side — in this case, `Equipment` should hold a reference to `Laboratory`, not the other way around.

Why it matters: Placing `Equipment_ID` in `Laboratory` implies that each laboratory has at most one piece of equipment (since a single cell cannot store multiple values without violating 1NF). This completely misrepresents the $1 : M$ cardinality. If we tried to store multiple equipment IDs, we would again violate 1NF.

Why I made this mistake: I may have been thinking of the relationship as “a laboratory owns equipment,” and then placed the ownership pointer in the laboratory tuple. This is the direction a programmer might naturally think (“store the child’s ID in the parent”), but it is backwards for relational modelling.

Fix: `Equipment_ID` is removed from `Laboratory`. The `Equipment` relation already correctly carries `{Lab_Name, Dept_Name}` as part of its composite PK and as a FK to `Laboratory`, which captures the ownership semantics correctly.

4.5 Revision 5: Comments_Suggestion Embedded in Stakeholder

What was wrong: I included `Comments_suggestion_date` and `comments_suggestion_topic` as plain attributes of `Stakeholder`. The ER diagram explicitly marks `Comments_Suggestion` as a *weak entity* of `Stakeholder`, with its own partial key `{Date, Time}`. A weak entity must be modelled as a separate relation.

Why it matters: Embedding comments in `Stakeholder` implies a one-to-one relationship: each stakeholder has at most one comment. This directly contradicts the $1 : M$ cardinality in the ER

diagram (1 `Stakeholder` assigns M `Comments_Suggestions`). Additionally, the topic/content of a comment is not a property of the stakeholder themselves, so embedding it conflates two distinct real-world concepts.

Why I made this mistake: I likely recognised that the comment was associated with the stakeholder and collapsed it in, the way one might record a “primary comment” field on a user profile. I may have been rushing and missed the weak-entity double-diamond notation on the ER diagram. The fact that I did not write a 3NF conclusion for `Stakeholder` suggests I noticed something was incomplete but ran out of time.

Fix: I created a separate `Comments_Suggestion(Person_ID, Date, Time, Content)` relation. The composite PK `{Person_ID, Date, Time}` reflects the weak entity’s dependence on its owner and the partial key from the ER diagram, while allowing each stakeholder to have an arbitrary number of comments.

4.6 Revision 6: Missing Relationship Relations (1NF and Semantic Completeness)

What was wrong: Beyond the multi-valued attribute problem discussed in Revision 4.3, the deeper issue is that several explicit ER relationships were not translated into relational tables at all:

- `Student` takes `Course` ($M : N$) — entirely absent.
- `Undergrad` uses `Teaching_Laboratory` ($M : N$) — entirely absent.

Why it matters: Every $M : N$ relationship in an ER diagram *must* become its own relation in the relational schema. There is no other valid way to represent it without violating 1NF or introducing massive redundancy.

Fix: I added `Takes(Student_ID, Cid)` and `Uses(Student_ID, Lab_Name, Dept_Name)` as separate relations.

4.7 Revision 7: Equipment Incorrectly Flagged as Not in 3NF

What was wrong: I wrote “Not in 3NF” for `Equipment` without any explanation. The FD I stated was:

$$\{ID, Dept_Name, Lab_Name\} \rightarrow \{Model_No, Date_Purchased, Name\}$$

With the PK `{ID, Dept_Name, Lab_Name}`, all non-key attributes depend on the entire key with no partial or transitive dependencies. This relation satisfies BCNF, which implies 3NF.

Why it matters: Falsely claiming a relation is not normalised when it is leads to unnecessary decomposition or, worse, signals a fundamental misunderstanding of what constitutes a normalization violation.

Why I made this mistake: I believe I assumed that any weak entity relation with a composite PK automatically has normalization problems — perhaps associating “weak entity” with “complicated” or “problematic.” That is a misconception. A weak entity simply requires a composite PK including the owner’s key; as long as all non-key attributes depend on the full composite key with no transitive dependencies, it can be perfectly normalised.

Fix: In the revised schema, `Equipment` is correctly identified as being in BCNF, with an explicit justification checking both partial and transitive dependencies.

4.8 Revision 8: Laboratory's 3NF Status and the DeptNum Dependency

What was wrong: I flagged Laboratory as not in 3NF but provided no reasoning. Upon reflection, the 3NF violation arises specifically from assumption A3: $\text{Dept_Name} \rightarrow \text{DeptNum}$. Let us analyse this formally.

Given the PK {Lab_Name, Dept_Name}, the attribute Dept_Name is a *prime attribute*. The attribute DeptNum is a *non-prime attribute*. The FD is:

$$\text{Dept_Name} \rightarrow \text{DeptNum}$$

Here, Dept_Name is *not* a superkey (it does not determine Lab_Name), and DeptNum is *not* a prime attribute. Therefore, this FD constitutes a 3NF violation (transitive dependency of a non-prime attribute on the key, routed through a prime attribute). It also clearly violates BCNF since Dept_Name is not a superkey.

Fix: I decomposed Laboratory by extracting the violating dependency:

- Department (Dept_Name, DeptNum) — captures the department-number mapping.
- Laboratory (Lab_Name, Dept_Name, Capacity, Location, Lab_Type) — the remainder, with Dept_Name as a FK to Department.

The decomposition is lossless-join (we can reconstruct the original by joining on Dept_Name) and dependency-preserving.

4.9 Revision 9: Missing Person_ID Foreign Keys in Subtype Relations

What was wrong: My original design listed Staff_ID, Student_ID, and Professor_ID as standalone attributes with no connection back to the Person supertype. Since the ER diagram uses an ISA hierarchy rooted at Person, each subtype entity IS a person. The shared Person_ID must appear in each subtype relation both as its own PK (or alongside a subtype-specific PK) and as a FK referencing Person.

Why it matters: Without this connection, the schema loses the ISA semantics entirely. There is no way to query, for example, “find the email address of all professors” by joining Professor and Person on Person_ID.

Fix: Every subtype relation (Staff, Student, Professor, Stakeholder) now includes Person_ID as both an attribute and a FK to Person(Person_ID). In cases where the subtype-specific ID (e.g., Professor_ID) is globally unique and serves as the PK, Person_ID forms a second candidate key.

4.10 Revision 10: Unjustified Dept_Name in Professor

What was wrong: I added Dept_Name to the Professor relation. This attribute does not appear in the ER diagram for the Professor entity. Introducing it without justification adds an unspecified FD and muddies the normalization analysis.

Why I made this mistake: In real universities, professors belong to departments, so it felt natural to include it. However, the ER diagram as given does not include this attribute for Professor, and I should not add attributes not supported by the diagram without explicitly stating and justifying the assumption.

Fix: Dept_Name is removed from Professor in the revised schema. If a professor-department relationship were needed, it would require either an explicit assumption or an additional relation, stated clearly.

5 Conclusion

This lab session gave me a concrete and sometimes humbling look at the gap between understanding relational theory conceptually and applying it precisely under real conditions.

My most pervasive mistake was confusing superkeys with candidate keys. I repeatedly listed non-minimal sets as candidate keys across every single relation. This is a foundational error because the entire framework of 3NF and BCNF is built on the correct identification of candidate keys. Going forward, I will always explicitly verify minimality before declaring a key to be a candidate key.

A second systematic error was the violation of 1NF through multi-valued attributes. I stored Course_list, teaching_courses, and supervising_topics as single attributes when they are clearly multi-valued. I now understand that any many-to-many relationship must become its own relation, and any one-to-many relationship from the “one” side must be captured by a FK on the “many” side, never by a list attribute.

Third, I underestimated the importance of properly modelling ISA hierarchies and weak entities. Collapsing them into type attributes or embedding weak entities in their owners are tempting shortcuts that lose essential relational content and semantic precision.

The Laboratory relation was the most instructive case for normalization theory. Correctly analysing it required distinguishing between prime and non-prime attributes and understanding why the 3NF definition allows an exception for prime attributes that BCNF does not. The Equipment case was also instructive: I incorrectly flagged it as not in 3NF when it was actually in BCNF, which taught me not to assume that structural complexity (e.g., a composite PK in a weak entity) implies a normalization problem.

Overall, the exercise reinforced that correct relational schema design requires careful, systematic reasoning rather than intuition alone, and that revisiting one's own work critically is an essential part of the design process.

References

- [1] William Ward Armstrong. 1974. Dependency Structures of Data Base Relationships. *Information Processing* 74 (1974), 580–583. <https://dl.acm.org/doi/10.5555/1229389.1229391>
- [2] Peter Pin-Shan Chen. 1976. The Entity-Relationship Model — Toward a Unified View of Data. *ACM Transactions on Database Systems* 1, 1 (1976), 9–36. doi:10.1145/320434.320440
- [3] Edgar F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387. doi:10.1145/362384.362685
- [4] Edgar F. Codd. 1971. *Further Normalization of the Data Base Relational Model*. Technical Report. IBM Research Laboratory. <https://dl.acm.org/doi/10.5555/800296.811680> Research Report RJ909.
- [5] Edgar F. Codd. 1974. Recent Investigations into Relational Data Base Systems. In *Proceedings of the IFIP Congress*. <https://dl.acm.org/doi/10.1145/800296.811691> IBM Research Report RJ1385. The source of the BCNF definition.
- [6] Ramez Elmasri and Shamkant B. Navathe. 2015. *Fundamentals of Database Systems* (7th ed.). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/fundamentals-of-database-systems/P200000003351>